Лабораторная работа 2. Ручное и автоматизированное тестирование

Имя студента: Цароев Альберт Казебкович
Группа: ПИЖ-б-о-23-1(1)
Предмет: Тестирование и отладка программного обеспечения
Претендую на оценку: Отлично

---

1. Описание метода и стратегии тестирования

1.1. Метод

Ручное тестирование - человек выполняет шаги тест-кейса и сам сравнивает результат с ожидаемым.
Ручная проверка хорошо подходит для внешнего вида, удобства и исследовательского тестирования. Её недостатки:
каждый прогон стоит времени, результат зависит от внимательности, а в игре ещё и от реакции (успеть
повернуть за 150 мс).

Автоматизированное тестирование - шаги и сравнение выполняет программа. Прогон дешёвый и одинаковый
каждый раз, поэтому его можно запускать после каждого изменения (регрессия). Минусы: нужен код тестов
и способ управлять приложением без человека.

Выбрано автоматизированное тестирование. Ручной чек-лист составлен первым: он фиксирует сценарии
так, как их проверяет игрок. Затем каждая строка чек-листа переведена в автотест.

1.2. Как игра управляется без человека

Автоматизировать игру с окном и таймером мешают две вещи: ввод идёт от оконной

системы, а ход змейки
зависит от реального времени. Обе решаются средствами Bevy, без изменений в коде игры:

Проблема	Решение в стенде `labs/common`
нужно окно и видеокарта	`MinimalPlugins` + `SnakeLogicPlugin`: логика работает без окна и рендера
ввод с клавиатуры	тест пишет сообщения `KeyboardInput`, те же, что шлёт окно; дальше их обрабатывает штатный `InputPlugin`
реальное время	`TimeUpdateStrategy::ManualDuration`: каждый кадр время сдвигается ровно на заданную величину; `tick()` проматывает время до следующего хода
случайная еда	`ScriptedSpawner` кладёт еду в заданные клетки

Тест читается как сценарий игрока:

```
let mut h = Harness::playing(config(20, 15), [Cell::new(11, 7)]);   // еда в 2 клетках пер
h.ticks(2);
assert_eq!(h.score(), 1);
assert_eq!(h.snake().len(), 4);
```

1.3. Стратегия

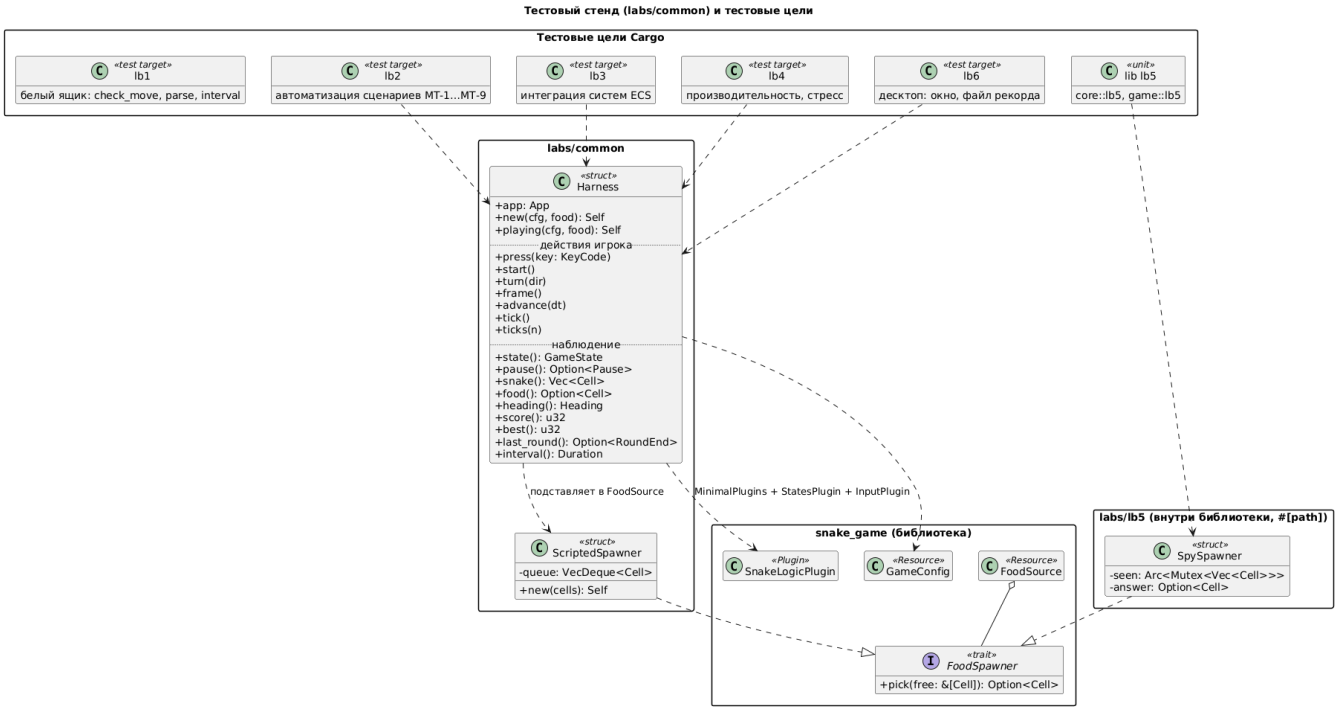
1. Составить ручной чек-лист по основным функциям (раздел 3.1) и один раз пройти его вручную.
2. Для каждой строки написать автотест с тем же номером (`mt5_eating_grows_and_scores` ↔ MT-5).
3. Строки, которые нельзя проверить без экрана (цвета, шрифт), остаются ручными. Их подкрепляют
   снимки экранов, которые делает `cargo run --example screens` с автопилотом.
4. Сравнить затраты на ручной и автоматический прогон.

1.4. Запуск

```
cargo test --test lb2
scripts/test.sh lb2 --report
cargo run --example screens        # снимки экранов → docs/screens/
```

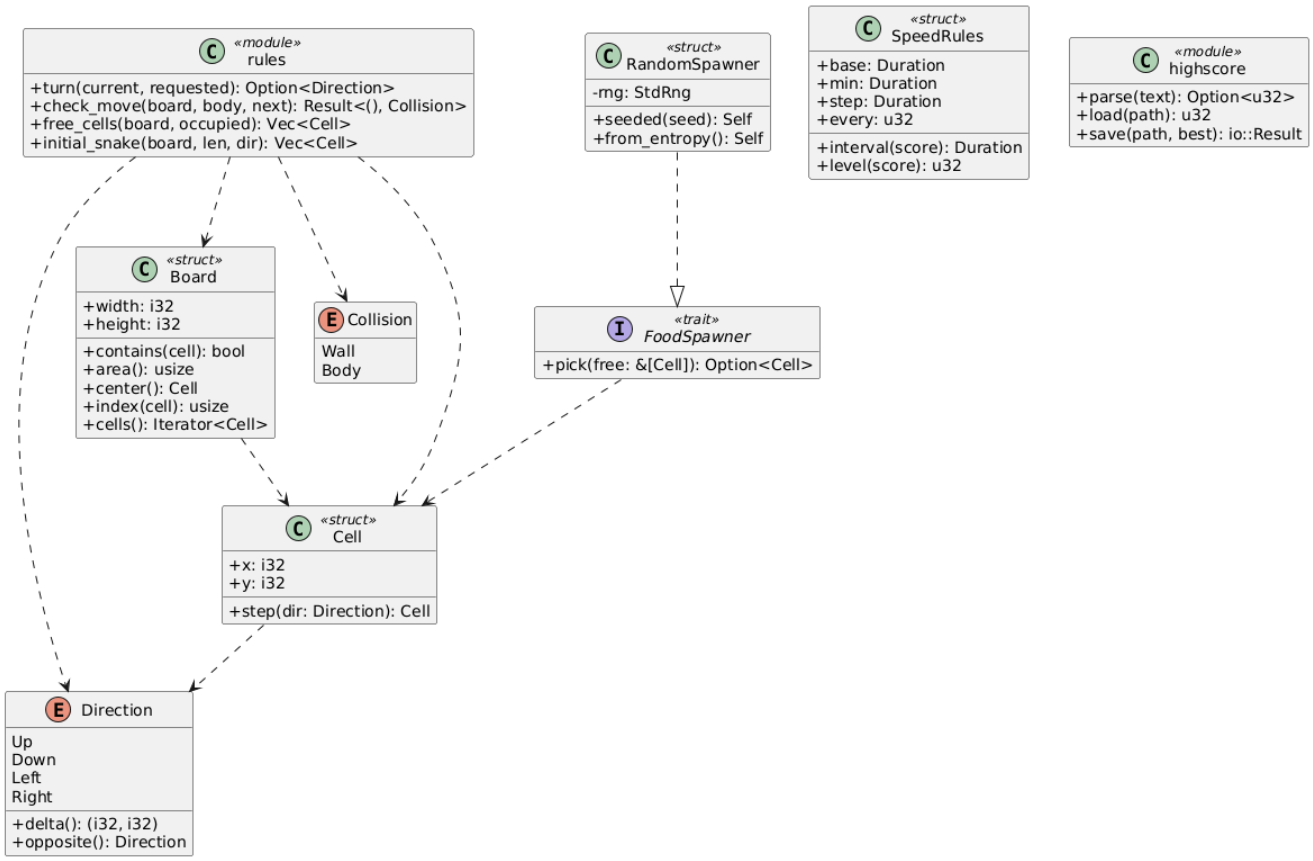
2. Диаграмма классов

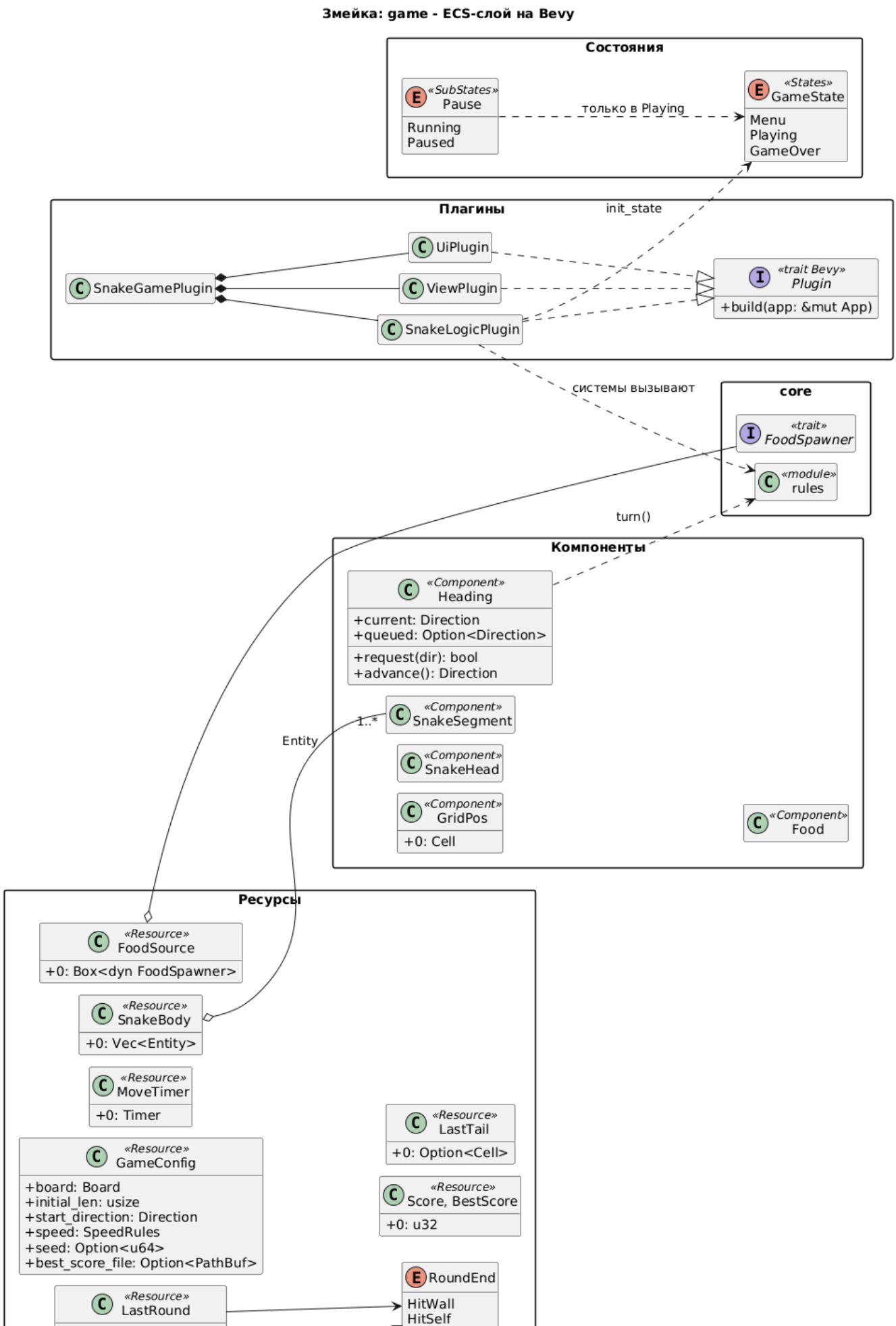
Тестовый стенд и его связь с игрой:



Классы игры:

Змейка: core - правила без Bevy




Змейка: game - ECS-слой на Bevy
Состояния
«SubStates»
Pause
Running
Paused
только в Playing
«States»
GameState
Menu
Playing
GameOver
Плагины
init_state
UiPlugin
SnakeGamePlugin
ViewPlugin
SnakeLogicPlugin
«trait Bevy»
Plugin
+build(app: &mut App)
системы вызывают
core
«trait»
FoodSpawner
«module»
rules
turn()
Компоненты
«Component»
Heading
+current: Direction
+queued: Option<Direction>
+request(dir): bool
+advance(): Direction
1..*
Entity
«Component»
SnakeSegment
«Component»
SnakeHead
«Component»
GridPos
+0: Cell
«Component»
Food
Ресурсы
«Resource»
FoodSource
+0: Box<dyn FoodSpawner>
«Resource»
SnakeBody
+0: Vec<Entity>
«Resource»
MoveTimer
+0: Timer
«Resource»
LastTail
+0: Option<Cell>
«Resource»
GameConfig
+board: Board
+initial_len: usize
+start_direction: Direction
+speed: SpeedRules
+seed: Option<u64>
+best_score_file: Option<PathBuf>
«Resource»
Score, BestScore
+0: u32
RoundEnd
HitWall
HitSelf
«Resource»
LastRound

3. Тестовые материалы

Код автотестов: scenarios.rs. Стенд: labs/common/mod.rs.

3.1. Ручной чек-лист и соответствие автотестам

Начальное положение: поле 20×15, змейка из 3 клеток, голова в (9,7), движение вправо.

№	Шаги	Ожидаемый результат	Автотест
MT-1	Запустить игру, нажать Enter	Меню исчезает, на поле змейка из 3 клеток и еда, счёт 0	`mt1_enter_starts_round`
MT-2	Ничего не нажимать	Змейка смещается на 1 клетку за ход, между ходами стоит	`mt2_snake_moves_one_cell_per_tick`
MT-3	Нажать ↑	На следующем ходу змейка идёт вверх	`mt3_arrow_turns_snake`
MT-4	Двигаясь вправо, нажать ←	Нажатие игнорируется, змейка продолжает вправо	`mt4_reverse_is_ignored`
MT-5	Направить змейку на еду	Счёт +1, змейка длиннее на 1, еда появилась в другом месте	`mt5_eating_grows_and_scores`
MT-6	Нажать P, подождать,	На паузе змейка стоит, после -	`mt6_pause_freezes_and_resumes`

№	Шаги	Ожидаемый результат	Автотест
	нажать Esc	продолжает	
MT-7	Довести змейку до стены	Экран «Игра окончена», причина «Столкновение со стеной»	`mt7_wall_ends_round`
MT-8	На экране конца партии нажать Enter	Новая партия: счёт 0, змейка в начальном положении	`mt8_enter_restarts_after_game_over`
MT-9	На паузе нажать M	Меню, поле пустое	`mt9_menu_from_pause`
MT-10	Посмотреть на экраны	Кириллица читается, голова светлее тела, еда красная	ручная, снимки `docs/screens/`
MT-11	Навести мышь на кнопку и нажать	Кнопка подсвечивается, действие выполняется	ручная

3.2. Входные данные

Тест	Конфигурация	Еда (сценарий)
MT-1…MT-4, MT-6, MT-7, MT-9	20×15, длина 3, вправо	(0,0), вдали от маршрута
MT-5	20×15	(11,7), на пути змейки

Тест	Конфигурация	Еда (сценарий)
MT-8	20×15	(10,7), съедается на первом ходу

3.3. Сравнение затрат

	Ручной прогон MT-1…MT-9	Автоматический
Время	*замерить при ручном прогоне* (минуты)	0.02 с (см. протокол)
Повторяемость	зависит от реакции: повернуть точно в клетку за 150 мс трудно	одинаковый результат каждый раз
Что видно	всё на экране, включая внешний вид	только то, что проверяет `assert`

4. Результаты выполнения тестов

Все 9 автотестов пройдены. Протокол: results.txt.

```
Протокол: lb2
Система:  Linux 7.0.0-38-generic x86_64
Rust:     rustc 1.98.1 (48a229cea 2026-09-01)
Команда:  cargo test --test lb2

test scenarios::mt1_enter_starts_round ... ok
test scenarios::mt2_snake_moves_one_cell_per_tick ... ok
test scenarios::mt3_arrow_turns_snake ... ok
test scenarios::mt4_reverse_is_ignored ... ok
test scenarios::mt5_eating_grows_and_scores ... ok
test scenarios::mt6_pause_freezes_and_resumes ... ok
test scenarios::mt7_wall_ends_round ... ok
test scenarios::mt8_enter_restarts_after_game_over ... ok
test scenarios::mt9_menu_from_pause ... ok

test result: ok. 9 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Ручной прогон МТ-10, МТ-11: артефактов не наблюдается

Меню	
